Lifeline or trap? The consequences of "vibe coding" and excessive reliance on AI

1st Author's Name	2nd Author's Name	3nd Author's Name
Krémer Aliz	Hősi Rezső Rudolf	Oláh Sára
ELTE	ELTE	ELTE
a0tyh8@inf.elte.hu	dov7lk@inf.elte.hu	a4mrc6@inf.elte.hu

Abstract – Context: Systematic reviews are important tools for researchers to get along with the state of art in their area of interest. Goal: This template aims to describe the general structure of a systematic review report, suggesting topics, contents and other aspects that the authors should pay attention when reporting their study. Method: This abstract has been written following the structure abstract style, which we strongly suggest to the authors. Results: The abstract should be written last and very carefully. The authors should have synthesis in mind when writing the abstract. Conclusion: Before writing your abstract, make sure you have read several other systematic review papers, and that you recognize what the most important information is to put here in the abstract. *(Abstract)*

Keywords - Vibe coding; Debugging Paradox; Programming education; LLMs, Cognitive offloading, Over-reliance

1 INTRODUCTION

1.1 The Rise of "Vibe Coding" and AI-Assisted Development

The rapid advancement of Large Language Models (LLMs) has initiated a significant paradigm shift in software engineering and computing education. By generating functional code from natural language descriptions, these AI tools have drastically lowered the barrier to entry for programming. This technological leap has given rise to a new programming paradigm colloquially known as "vibe coding". In vibe coding, developers express their software ideas in natural language, relying on the AI to interpret these high-level intents and generate the corresponding executable code. This approach shifts the developer's role from a traditional coder focused on syntactic precision to a conceptual guide. While this democratization expands access to computational tools for non-experts, it fundamentally alters the cognitive demands of programming, shifting the human effort from writing code to reading, comprehending, evaluating, and repairing AI-generated outputs. [Fortes-Ferreira et al., 2025], [Kazemitabaar et al., 2023], [Lau and Guo, 2023]

1.2 The Research Problem: Debugging as the Critical Bottleneck

Although vibe coding significantly accelerates the initial code generation phase, it introduces severe challenges later in the software development lifecycle, particularly for novices. We identify this core issue as the "Debugging Paradox": the very tools that provide a fast initial start ultimately leave beginners helpless when the AI-generated code contains complex, undocumented, architectural, or logical errors. While LLMs excel at fixing superficial syntax errors, debugging requires a deep validation of the causal relationships between the code and the overall system logic. Because the students did not construct the code step-by-step through their own cognitive effort, they lack the essential prior knowledge and structural understanding required to troubleshoot it. Consequently, when the AI hallucinates, generates incorrect control flows, or fails to resolve an environment-specific issue (e.g., a dependency mismatch), students are left stranded with a codebase they do not understand, leading to frustration and an inability to manually resolve the problem. [Prather et al., 2024]

1.3 Research Objectives and Hypotheses

To systematically investigate the "Debugging Paradox" and understand how vibe coding affects novice programmers' cognitive processes and problem-solving resilience, this study proposes eight hypotheses. These hypotheses are categorized into three core dimensions of the AI-assisted software development lifecycle.

1.3.1 Learning Phase and Cognitive Integrity

- **H1 (Superficial Learning):** Novice programmers using AI tools bypass the explicit, deep learning phase. By utilizing the AI as a shortcut, they fail to engage in the authentic problem-solving processes required for skill acquisition.
- **H2 (Lack of Comprehension):** Students frequently utilize AI-generated code without understanding its underlying mechanics. Uncritical acceptance leads to an inability to recognize even basic structural errors, such as infinite loops, within their own code.
- **H6 (The Prompt-Looping Anti-Pattern):** When the AI fails or makes an error, students disproportionately attempt to bypass the issue with new prompts ("prompt-looping") rather than manually reading and interpreting the codebase.

1.3.2 Debugging Dynamics and Interaction

- **H3 (Debugging Inability):** Due to superficial learning (H1) and a lack of comprehension (H2), students are left unequipped to identify and fix bugs. This becomes especially evident when dealing with superficially functional but logically fragile "house of cards" code generated by AI.
- **H4 (Over-Reliance):** Novice programmers fall into the trap of blind trust and over-reliance on AI-generated solutions.

1.3.3 Competence Gap and Systems Thinking

- **H5 (Architectural Confusion):** As application size and complexity increase, beginners lacking background knowledge struggle to integrate independently generated AI snippets. This results in an inconsistent and fragile "patchwork codebase".
- **H7 (Absence of Mental Models):** Complete cognitive offloading to AI prevents the formation of a coherent mental model of the code. Consequently, students are rendered incapable of identifying and manually validating complex, intentionally placed errors (e.g., outdated library dependencies).
- **H8 (Skill-Based Divide):** Programmers with better foundational skills and higher self-efficacy recognize AI limitations earlier, abandon prompt-looping, and successfully transition to manual debugging. In contrast, weaker students exhibit over-reliance and prematurely abandon the task.

2 LITERATURE REVIEW

2.1 Cognitive Offloading and the Erosion of Skill Formation

The phenomenon of delegating cognitive processes to external tools to reduce mental effort is known in psychology as cognitive offloading. In the context of programming education, AI code generators theoretically reduce extraneous load, allowing learners to focus on higher-level algorithmic problem-solving. However, recent empirical studies suggest a contrary outcome. Relying heavily on AI to complete tasks quickly often comes at the cost of real skill development, particularly for junior developers who fail to remain cognitively engaged. Because students use AI as a shortcut, they bypass the explicit learning phase, which impairs their conceptual understanding and code reading abilities (supporting H1). Furthermore, novices using Generative AI frequently skip crucial problem-solving planning stages and jump straight into implementation, a metacognitive difficulty identified as "Location", which is exacerbated by AI tools that entice them. [Shen and Tamkin, 2026], [Prather et al., 2024]

2.2 The Illusion of Competence and Metacognitive Difficulties

This superficial engagement fosters a dangerous false sense of security. Novice programmers often lack the metacognitive awareness and mental models required to evaluate the code they generate. When working with Generative AI, struggling students frequently experience cognitive dissonance and maintain an unwarranted "illusion of competence" because the tool gives them a false sense of progress. Prather et al. identified specific novel metacognitive difficulties arising from AI use, such as "Progression," where students are conceptually behind in the course material but remain unaware of it due to their false confidence in the generated output. Additionally, students often face the "Mislead" difficulty, where the AI leads them down the wrong path, and they blindly follow because they accept suggestions without critical evaluation (supporting H2 and H4). [Prather et al., 2024]

2.3 Vibe Coding, "House of Cards" Architecture, and Prompt-Looping

These cognitive deficiencies culminate in the emerging practice of "vibe coding". While this intent-based approach accelerates initial development, it frequently produces what industry experts call "house of cards code"—implementations that function superficially but harbor critical deficiencies in error handling, performance optimization, and logic. Because novices assemble these disjointed AI responses without a holistic architectural understanding, the result is a fragile and inconsistent "patchwork codebase". When this codebase inevitably fails or produces logical errors, novices are unequipped to debug it. They lack fundamental debugging skills, leading to a significant gap in performance specifically related to troubleshooting. Instead of pausing to systematically trace the logic, students experience the "Interruption" difficulty—an inability to concentrate on problem-solving due to frequent and sometimes unhelpful AI code suggestions. Consequently, they fall into the "prompt-looping" anti-pattern, repeatedly throwing errors back at the AI or randomly modifying prompts without manually reading the codebase (supporting H3, H5, and H6). [Maes, 2025], [Shen and Tamkin, 2026], [Prather et al., 2024]

2.4 The Competence Gap and Classical Quality Assurance

Ultimately, the integration of generative AI in programming widens the digital divide among learners. While weaker students exhibit severe over-reliance and premature abandonment, skilled programmers interact with AI fundamentally differently. Participants with higher grades and higher self-efficacy are significantly less likely to experience GenAI-induced metacognitive difficulties. Research by Kusper et al. on Hungarian informatics students further demonstrates this skill-based divide: better programmers evaluate AI-generated code much more critically. Higher-competence students consistently maintain classical quality assurance behaviors—they actively review, debug, and strive to understand the underlying mechanics of AI-generated code before executing it. This highlights that foundational skills are prerequisites for successfully transitioning from AI-assisted generation to manual debugging, preventing the debugging paradox (supporting H7 and H8). [Prather et al., 2024], [Kusper et al., 2025]

3 METHODOLOGY

(Vázlat)
4.1. Kísérleti setup: A „szándékos hiba"

- A feladat: Egy egyszerű API integráció, ahol egy elavult könyvtárverzió miatt a „hallucinált" vagy generált kód futásidejű hibát eredményez.
- A hiba jellege: Nem szintaktikai, hanem logikai/környezeti, amit az AI nem lát át közvetlen kontextus nélkül.

4.2. Mért változók és mérőszámok

- Persistence metrika: Hány egymást követő promptot küld a felhasználó anélkül, hogy manuálisan módosítaná a kódot vagy elolvasná a dokumentációt.
- Latency to Manual Shift: Az az idő, ami eltelik az első hibaüzenet és az első manuális debug-lépés (pl. console.log vagy debugger használata) között.
- Frusztrációs skála: Likert-skálás kérdőív a „trial-and-error loop" okozta stressz mérésére.

4 RESULTS

(Vázlat)
A hipotéziseket három logikai klaszterbe soroljuk a szoftverfejlesztési életciklus alapján:
I. Tanulási fázis és Kognitív integritás (H1, H2, H6)

- H1-H2 (Kognitív erózió): Az adatok várhatóan alátámasztják, hogy a „vibe coding” során elmarad a kód soronkénti értelmezése, ami akadályozza a mély tanulást.
- H6 (Alaptudás szerepe): Azok a fejlesztők, akik rendelkeznek korábbi manuális tapasztalattal, hamarabb felismerik az AI korlátait (mentőöv szerep).

II. Hibakeresési fázis és Interakciós dinamika (H3, H4)

- H3-H4 (A Debugolási Csapda): A kísérleti csoport hajlamos a „prompt-loop”-ra (ugyanazt a hibát többször visszadobja az AI-nak), ahelyett, hogy elolvasná a hibaüzenetet (manuális kódolvasás elkerülése).
- Metakognitív monitoring: A résztvevők tévesen ítélik meg saját haladásukat a generált kód mennyisége alapján.

III. Kompetencia-szakadék és Rendszerszemlélet (H5, H7, H8)

- H5 (Architekturális vakság): A kód „patchwork” jellege miatt a fejlesztő nem látja át a modulok közötti függőségeket.
- H7-H8 (Over-reliance): A túlzott hagyatkozás korrelál a kritikai gondolkodás és a logikai következtetés (inference) csökkenésével.

5 DISCUSSION

(Vázlat)
6. Diszkusszió (Discussion) 6.1. A „Patchwork Architecture” veszélyei

- A szoftver nem egy koherens egész, hanem egymáshoz „ragasztott” AI-válaszok összessége. Ez karbantarthatatlan kódbázishoz vezet.

6.2. A „Vibe Debugging” korlátai

- A debugolás során a fejlesztő nem a logika, hanem a „vibe” (hasonlóság) alapján próbálja javítani a kódot, ami a statisztikai valószínűségre épít a kauzalitás helyett.

6.3. Pedagógiai javaslatok

- „AI-free” alapozó szakaszok: A debugolási alapok (stack trace olvasás, memory heap elemzés) tanítása AI segítsége nélkül.
- Prompt-kritika: A hallgatóknak nem csak kódot kell generálniuk, hanem meg kell indokolniuk az AI által hozott döntéseket.

6 CONCLUSION

(Vázlat)
A kutatás összegzése: Miközben az AI drasztikusan csökkenti a belépési küszöböt a programozás világába, egyúttal egy „üvegplafont” is létrehoz. A fejlesztők képesek komplex rendszereket felépíteni, de képtelenné válnak azok javítására, amint az AI kontextusa vagy tudása véget ér. Ez a paradoxon a szoftverminőség romlásához és a fejlesztői autonómia elvesztéséhez vezethet.

References

[Fortes-Ferreira et al., 2025] Fortes-Ferreira, M., Alam, M. S., and Bazilinskyy, P. (2025). Vibe coding in practice: Building a driving simulator without expert programming skills. In *Adjunct Proceedings of the 17th International Conference on Automotive User Interfaces and Interactive Vehicular Applications*.

[Kazemitabaar et al., 2023] Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., and Grossman, T. (2023). Studying the effect of ai code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*.

[Kusper et al., 2025] Kusper, G., Mátyás, G. I., and Balla, T. (2025). We are not afraid of the wolf! – ai usage attitudes among hungarian informatics students. *Annales Mathematicae et Informaticae*, 61:186–201.

[Lau and Guo, 2023] Lau, S. and Guo, P. J. (2023). From "ban it till we understand it" to "resistance is futile": How university programming instructors plan to adapt as more students use ai code generation and explanation tools such as chatgpt and github copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1*, pages 106–121.

[Maes, 2025] Maes, S. H. (2025). The gotchas of ai coding and vibe coding. it's all about support and maintenance. Zenodo.

[Prather et al., 2024] Prather, J., Reeves, B., Leinonen, J., MacNeil, S., Randrianasolo, A. S., Becker, B., Kimmel, B., Wright, J., and Briggs, B. (2024). The widening gap: The benefits and harms of generative ai for novice programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research*.

[Shen and Tamkin, 2026] Shen, J. and Tamkin, A. (2026). How ai impacts skill formation. *arXiv preprint arXiv:2601.20245*.